MonoSLAM

MonoSLAM is a monocular EKF SLAM, after Davison et al. [3]: one camera, no other sensor, and a single extended Kalman filter for the camera and the map. It is initialized from a chessboard, which fixes the scale, and then runs on the natural features of the scene. This note describes the system as it is: the frame loop (section 1), the state (section 2), the motion model (section 3), the features (section 4), how they are found (sections 5 and 6), the update (section 7), the map (section 8), how it is found again when tracking fails (section 9), and how well it works (section 10).

1 The frame loop

The camera is asked for 320×240 frames, which it gives at about 100 Hz with a fixed exposure of 5 ms, and the filter works on them as they are (`FrameSource`). Every length in pixels below is one of these frames, also those measured at 640×480 before, halved. With a quarter of the pixels of 640×480, the frames cost about half the time, with the same accuracy (section 10). The calibration is fitted on the largest frames the camera gives in the same shape, 1024×768, where the corners are the most precise, and saved for the small ones: a pixel center x becomes $(x + \frac{1}{2})s - \frac{1}{2}$, with s the shrinking factor, and the distortion, in normalized coordinates, does not change.

Every frame goes through the same steps, in `main.cpp`:

1. `ekf.predict(dt)` moves the camera with the motion model, over the real interval between the frames, from their capture times. Live, frames are dropped while one is processed, so the interval can be several frame periods.

2. `tracker.predict(gray)` moves the *seed* of every feature, where it is expected in the new image, with the optical flow (section 5).

3. After a weak frame (section 9), `keyframes.match` looks for the map where the keyframes that look like the frame saw it, and if that fails, `matchGlobally` anywhere in the frame by its appearance. If either finds it, the camera is moved there, the seeds of the features found to where they were found, and the others to the projections of their features.

4. `ekf.observe()` predicts the features in view, `matchFeatures` searches each of them around its seed, or around its prediction if it is coming back into view (section 6), and `ekf.update` decides which matches to use, with 1-point RANSAC (section 7).

5. If at least four features were used, the seeds move to the projections of their features from the updated state, and otherwise they keep following the images.

6. Unless the frame is weak, the map is kept: the features rarely matched are removed, and new ones are added where the frame has none (section 8), and the frame becomes a keyframe if it was taken far from every other one.

The information flows in a loop: images → seeds → where to search → matches → filter → projections → seeds. The filter closes it in step 5, which keeps the seeds from drifting, and the flow keeps it going in step 2 when the filter cannot.

step	reads	writes
`ekf.predict`	state	state
`tracker.predict`	images, seeds	seeds
`Keyframes::match`, `matchGlobally`	image, state, keyframes	state, seeds
`matchFeatures`	image, predicted state, seeds	matches
`ekf.update`	state, matches	state
`tracker.correct`	state	seeds
`Mapper`	image, state	state, seeds
`Keyframes::add`	image, state	keyframes

Everything the filter knows comes through its own models and its own update. The seeds only decide where to look, and the only step that sets the camera without an update is the relocalization of step 3.

2 State and conventions

The state is

$$x = \begin{pmatrix} r \\ e \\ v \\ \omega \\ y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{12+6N}, \tag{1}$$

where $r \in \mathbb{R}^3$ is the camera position in world coordinates, $e \in \mathbb{R}^3$ an orientation error (below), v the linear velocity in world coordinates, ω the angular velocity in camera coordinates, and $y_i \in \mathbb{R}^6$ the features in inverse depth (section 4). The world is the frame of the chessboard, which lies on the plane $z = 0$. Lengths are in cm, angles in rad.

The orientation. The orientation is a rotation matrix R, the change of basis from camera to world coordinates: a point with camera coordinates X_c has world coordinates

$$X_w = R\,X_c + r, \tag{2}$$

so the columns of R are the camera axes in world coordinates, and $R_{cw} = R^\top$ goes from world to camera. R is not in the state vector: the filter keeps it next to it, and the state holds a small rotation e of the camera on top of it, the orientation being

$$R\,\exp([e]_\times), \tag{3}$$

with $\exp([e]_\times)$ the rotation by $\|e\|$ radians about e (Rodrigues' formula, `rotation`) and $[e]_\times$ the matrix of the cross product (`skew`). P is the uncertainty of e, not of R. After every update R absorbs e, $R \leftarrow R\exp([e]_\times)$, and e goes back to zero, so all the Jacobians are taken at $e = 0$, where $\exp([e]_\times) \approx I + [e]_\times$. This is the error-state, or multiplicative, EKF [5].

When R absorbs e, the uncertainty of the error moves to the new reference too: an error d about $R\exp([e]_\times)$ is one of $G\,d$ about R, with $G = I - \frac{1}{2}[e]_\times$, so the orientation rows and columns of

P are multiplied by G. It looks negligible for the small e of one update, but it is first order in e, and without it the runs drift apart within ten frames from those of a quaternion, which does it implicitly.

Unlike a quaternion, e has as many entries as the orientation has degrees of freedom, so there is no unit norm to keep and P is not singular. Unlike a rotation vector of the whole orientation, e stays small, far from the angle of π where that wraps around and its covariance means nothing.

Composing rotations. When the camera rotates there are two camera frames, the previous one C and the new one C'. The new orientation $R' = R\exp([w]_\times)$ gives

$$X_w = R'\,X_{c'} + r = R\,\exp([w]_\times)\,X_{c'} + r, \tag{4}$$

a chain of two changes of basis: $\exp([w]_\times)$ maps coordinates in C' to coordinates in C, and R maps those to the world. The axis w has the same coordinates in C and C', since the rotation fixes it, so w is a rotation in camera coordinates. In the world it is $R\,w$, and $R\exp([w]_\times) = \exp([Rw]_\times)R$. The motion model composes on the right, so ω is in camera coordinates, and so is e.

The right Jacobian. A small change d of the rotation vector of $\exp([\theta]_\times)$ is, to first order, a small rotation in its own frame,

$$\exp([\theta + d]_\times) \approx \exp([\theta]_\times)\exp([J_r(\theta)\,d]_\times), \qquad J_r(\theta) = I - \frac{1-\cos\varphi}{\varphi^2}[\theta]_\times + \frac{\varphi - \sin\varphi}{\varphi^3}[\theta]_\times^2, \tag{5}$$

with $\varphi = \|\theta\|$ (`rightJacobian`). $J_r(0) = I$, and for the rotation of one frame, $\varphi \approx 0.05$, it differs from I by a few percent.

3 The motion model

Between two frames the velocities change by an unknown amount, the result of a random acceleration that is constant during the step: $V = a\,\Delta t$ and $\Omega = \alpha\,\Delta t$, with $a \sim \mathcal{N}(0, \sigma_a^2 I_3)$ and $\alpha \sim \mathcal{N}(0, \sigma_\alpha^2 I_3)$. The velocities also decay towards zero, with time constants τ_v and τ_ω. With $e_v = e^{-\Delta t/\tau_v}$ and $h_v = \tau_v(1 - e_v)$, and likewise e_ω and h_ω,

$$\begin{aligned} r_{k+1} &= r_k + h_v\,v_k + \tfrac{1}{2}V\,\Delta t, \\ R_{k+1} &= R_k \exp\left(\left[h_\omega\,\omega_k + \tfrac{1}{2}\Omega\,\Delta t\right]_\times\right), \\ v_{k+1} &= e_v\,v_k + V, \\ \omega_{k+1} &= e_\omega\,\omega_k + \Omega, \\ y_{i,k+1} &= y_{i,k}, \end{aligned} \qquad Q = \Delta t^2 \begin{pmatrix} \sigma_a^2 I_3 & 0 \\ 0 & \sigma_\alpha^2 I_3 \end{pmatrix}, \tag{6}$$

where Q is the covariance of (V, Ω) (`MotionModel`).

The decay. h is the distance a decaying velocity covers in a step, divided by the velocity at its start. For $\tau \to \infty$, $e \to 1$ and $h \to \Delta t$, which is the constant velocity model of Davison et al. [3], and for $\tau \to 0$ the velocity is forgotten at once and the camera is predicted to stay where it is. A hand-held camera keeps moving from one frame to the next, but not for long in the same way, and the decay also bounds how far the prediction drifts when nothing is measured.

The factor $\frac{1}{2}$. With an acceleration that is constant during the step, the velocity changes linearly from v_k to $v_k + V$, and the position moves by the mean velocity of the step, $r_{k+1} = r_k + (v_k + \frac{1}{2}V)\,\Delta t$. The model of Davison et al. [3] uses $(v_k + V)\,\Delta t$, an impulse at the start of the step. Both give the same predicted mean, and differ in how the noise reaches the position: a variance of $\frac{1}{4}\sigma_a^2\Delta t^4$ instead of $\sigma_a^2\Delta t^4$. The same factor is applied to the rotation, which is exact about a fixed axis and correct to first order otherwise.

Linearization. The prediction is linearized at the mean angular velocity, which is not zero. With $\theta = h_\omega\,\omega_k$, the orientation becomes $R_{k+1} = R_k \exp([\theta]_\times)$, and the error after the step is e_{k+1} with

$$R_{k+1}\exp([e_{k+1}]_\times) = R_k\exp([e_k]_\times)\exp([\theta + h_\omega\,d\omega]_\times), \tag{7}$$

for an error $d\omega$ of the angular velocity. With the right Jacobian, and moving $\exp([e_k]_\times)$ across $\exp([\theta]_\times)$ as above, $e_{k+1} \approx \exp(-[\theta]_\times)\,e_k + h_\omega J_r(\theta)\,d\omega$, so the Jacobian F of the camera part is the identity but for

$$\frac{\partial r_{k+1}}{\partial v_k} = h_v I_3, \quad \frac{\partial e_{k+1}}{\partial e_k} = \exp([\theta]_\times)^\top, \quad \frac{\partial e_{k+1}}{\partial \omega_k} = h_\omega J_r(\theta), \quad \frac{\partial v_{k+1}}{\partial v_k} = e_v I_3, \quad \frac{\partial \omega_{k+1}}{\partial \omega_k} = e_\omega I_3. \tag{8}$$

The noise enters through the velocities, so $W = \partial f/\partial(V, \Omega)$ is 12×6: $\frac{1}{2}\Delta t\, I_3$ and I_3 in the rows of r and v, and $\frac{1}{2}\Delta t\, J_r(\theta)$ and I_3 in those of e and ω. Only the camera moves, so P becomes

$$\begin{pmatrix} FP_{cc}F^\top + WQW^\top & FP_{cf} \\ P_{fc}F^\top & P_{ff} \end{pmatrix}, \tag{9}$$

with c the camera and f the features, and is symmetrized to keep the rounding errors of thousands of steps from adding up. The tests *Process Jacobian* and *Process noise Jacobian* check F and the orientation block of W against finite differences of the prediction, with the error measured from the nominal prediction.

Values. $\sigma_a = 1000$ cm/s^2, $\sigma_\alpha = 15$ rad/s^2, $\tau_v = 0.3$ s and $\tau_\omega = 0.2$ s. In one frame the velocities can change by about 33 cm/s and 0.5 rad/s, and without measurements they halve in about 0.2 and 0.14 s. The intervals come from the capture times of the frames, or are 1/30 s for recordings without them.

Why this model. The ground truth of the chessboard (section 10) gives the camera pose in every frame where the whole board is detected. A Kalman filter run on it with each candidate model, with the noise fitted by maximum likelihood over three sequences, scores how well the model predicts the next frame (negative log-likelihood, lower is better):

model	position	orientation
no velocity, a random displacement	1827	−10565
impulse [3]	674	−11655
constant acceleration during the step	649	−11710
same, with decay, $\tau = 10$ s	639	−11718
same, with decay, $\tau = 0.3$ s	595	−11858

Having the velocities matters most, by more than a thousand units: the motion of the camera persists from one frame to the next. How the noise enters them matters little, and a decay of about 0.3 s fits best, for one more parameter. The motion itself needs about 90 cm/s^2 and 5.5 rad/s^2, but the filter needs more, since it also has to absorb its own errors, and since the fastest frames, the most blurred, are those where the board is not detected and that this fit does not score.

4 Inverse depth features

A feature is the inverse depth parametrization of Civera et al. [1]:

$$y = (a, \theta, \phi, \rho) \in \mathbb{R}^6, \qquad p = a + \frac{1}{\rho}\, m(\theta, \phi), \qquad m(\theta, \phi) = \begin{pmatrix} \cos\phi \sin\theta \\ -\sin\phi \\ \cos\phi \cos\theta \end{pmatrix}, \tag{10}$$

where a is the anchor, the position of the camera when the feature was first seen, θ and ϕ the azimuth and elevation of the ray from a to the point, in world coordinates, and ρ the inverse of the distance along it.

Why. A point seen once is a ray: its depth is unknown until the camera has moved enough. Along the ray the uncertainty of the depth is very asymmetric (a point at 50 cm may be at 25 cm or at infinity), which a Gaussian on p cannot describe, but a Gaussian on ρ can: $\rho \in [0, 0.04]$ covers everything from 25 cm to infinity. The projection is also much closer to linear in ρ than in the depth. So a feature can be added to the filter at its first frame, with a large σ_ρ, and it constrains the orientation of the camera from then on, before its depth is known.

Observation.

$$h_c = R^\top \big(\rho\,(a - r) + m(\theta, \phi)\big) \tag{11}$$

is the point in camera coordinates scaled by ρ, which does not change its pixel, and is well defined at $\rho = 0$, for points at infinity. The pixel is the projection of h_c with the lens distortion of the camera, and the Jacobians are those of h_c with respect to r, e and y, times that of the projection. Rotating the camera by e turns h_c into $\exp(-[e]_\times)h_c \approx h_c + [h_c]_\times e$, so $\partial h_c / \partial e = [h_c]_\times$ (`inverse_depth::toCamera`, `CameraModel::projectCameraPoint`). A feature is in view if h_c is in front of the camera and its pixel inside the image.

No conversion to points. Civera et al. [1] convert a feature to a point $p \in \mathbb{R}^3$ once its depth is known well enough, to save three entries of the state. Here every feature keeps the same six parameters all the time, which keeps the observation model, the map and the code uniform, at the price of a slightly larger state. `EKF::isWellLocated` tells the features whose depth is known, $\rho > 0$ and $\sigma_\rho < 0.1\,\rho$, which is where Civera et al. would convert them. `inverse_depth::toPoint` gives the point when one is needed, for drawing or warping patches. A feature with $\rho \leq 10^{-3}$ is placed at 10 m along its ray.

Initialization. The camera pose comes from the whole chessboard, with PnP, if it explains all its corners within 3 px and 1 px RMS. Five of its corners, the four outermost and the one closest to the center, become the first features, at their known positions p_i, written with the anchor at the initial camera position, $y_i = (r_0, \theta_i, \phi_i, \rho_i)$ (`fromPoint`). The covariance of $(\theta_i, \phi_i, \rho_i)$ is $J\,(0.05\, I_3)\, J^\top$, with J the Jacobian of `fromPointJacobian`: a variance of 0.05 cm^2 per axis of the point. The anchor has no uncertainty: it is a constant, not something measured. The camera starts with standard deviations of 0.22 cm, 0.02 rad, 30 cm/s and 1.5 rad/s, and no velocity. This is where the scale of the map comes from, and the chessboard is not used afterwards.

New features. A feature first seen at pixel u from the pose (r, R), with an inverse depth ρ_0, is

$$y = \big(r,\ \theta(d),\ \phi(d),\ \rho_0\big), \qquad d = R\,\pi^{-1}(u), \tag{12}$$

where $\pi^{-1}(u)$ is the ray of the pixel in camera coordinates, found by undistorting it. Rotating the camera by e turns d into $R\exp([e]_\times)\pi^{-1}(u) \approx d - R\,[\pi^{-1}(u)]_\times e$. With $J = \partial y/\partial(r, e, u, \rho_0)$, of size 6×9 (`inverse_depth::fromPixel`), the state is augmented as

$$x \leftarrow \begin{pmatrix} x \\ y \end{pmatrix}, \qquad P \leftarrow \begin{pmatrix} P & P_{:,c}\,J_c^\top \\ J_c\,P_{c,:} & J_c\,P_{cc}\,J_c^\top + J_u\,\sigma_u^2\,J_u^\top + J_\rho\,\sigma_\rho^2\,J_\rho^\top \end{pmatrix}, \tag{13}$$

where c are the six entries of the camera position and orientation error, σ_u the pixel noise and σ_ρ that of the inverse depth (`EKF::addFeature`). The correlation with the pose is what makes the feature move with the camera when a later update corrects the pose at which it was added.

5 The seeds

The features have to be found in every new image, and the question is where to look. In EKF SLAM the search region is the 3σ ellipse of the innovation covariance around the prediction, the active search of Davison et al. [3]. That only works if the ellipse holds the feature and nothing that looks like it, and a hand-held camera jerks: on the first sequence the chessboard corners move up to 19 px between frames, and a constant velocity only reduces the median prediction error from 5 to 3 px, with 11 px at the 99th percentile. Searched as the active search does, the first two sequences are lost, in the frames where the motion changes by 10 px from one frame to the next. The motion model cannot predict that, but the images can.

`FlowTracker` follows the image motion of every feature, its *seed*, with no knowledge of the filter. It keeps the previous image and one seed per feature, including those out of view, and every frame (`predict`):

1. *Tracks generic corners.* The corners of the previous image are tracked to the new image and back with pyramidal Lucas–Kanade [6], six levels and a 11×11 window. A corner is kept if it comes back within 0.5 px of where it started, which removes most of those lost to blur or occlusion. The corners are kept from frame to frame, and only when fewer than 120 are left are new ones detected (`goodFeaturesToTrack`, quality 0.01, at least 4 px apart from each other and from those left), up to 150. Detecting them anew in every frame made the flow 40% slower. The six levels matter: with four, the second sequence is lost, because the flow cannot follow jumps of more than 50 px.

2. *Moves each seed with its neighbors.* An affine map is fitted with RANSAC to the motion of the 24 tracked corners closest to the seed (inliers within 1.5 px), and the seed is moved with it. With fewer than 6 corners, or if the fit fails, it stays. Seeds out of the image are moved the same way, by the corners closest to them.

3. *Lets the seed's own flow take over.* The seeds in view are also tracked themselves, with a 15×15 window. If that lands within 3 px of the affine position, it replaces it, since the flow of a single point is more precise. If not, it has probably slipped to a look-alike neighbor, and the affine position is kept.

An affine map is the first-order approximation of the image motion around a point, so this assumes that the seed moves like the corners around it, which holds for a locally smooth surface, and for any scene when the camera mostly rotates. At a depth discontinuity, with the camera translating, the fit follows whichever surface has more corners. The test *Flow follows depth layers* checks that two regions moving apart each carry their own seeds.

After the update, `correct` moves the seeds to the projections of their features from the updated state, if at least four features were used, so that a seed that slipped is brought back by the others, through the pose. With fewer the pose is not well constrained, and the seeds keep following the images.

6 The patch search

Each feature keeps the patch, 11×11 px, of the frame where it was first seen. To find it in a new frame (`findPatch`), the patch is warped to how it would look from the predicted pose, taking it as a piece of the plane through the feature with its normal: the pattern's for the five chessboard corners, and facing the first camera for the others. The corners of the patch are mapped through the whole camera model, distortion included, and the homography is the one that maps them.

The warped patch is searched within ± 8 px of the seed, or of the prediction if the seed is out of view, by zero-mean normalized cross-correlation with the mask of the warped patch. The best peak is kept if it scores at least 0.6, and if no other peak, a local maximum more than 1 px away, scores within 0.05 of it: two near-equal peaks are a repeated texture or an edge, and which one wins then depends on tiny differences of the prediction. The peak is refined to subpixel precision with a parabola. Every feature found is returned, and the update decides which ones to use.

Returning features. A feature that was out of view for at least 10 frames is *returning* until the update uses it again. Its seed is no use: the flow could not follow it while it was away. It is searched around its prediction instead, within 3σ of the larger standard deviation of its innovation, as in the active search of Davison et al. [3], but no closer than ± 8 px and no wider than ± 20 px. While the camera is away its uncertainty grows, and so does the region, so the feature is found even if the map has drifted. The cap keeps a large uncertainty from inviting look-alikes: without it, at ± 30 px, some runs of sequence 6 end more than 3 m off. Over 15 runs, searching the returning features this way recovers 17 of the 30 kidnaps, against 9 when they were searched around their seeds.

7 The update

`EKF::observe` predicts the pixels h of the features in view, two rows per feature, their Jacobian H, which only has the columns of the camera and of the features in view, the innovation covariance $S = HPH^\top + \sigma^2 I$, with $\sigma = 1.5$ px, and the columns of P that H acts on. A pixel only depends on the camera and on its own feature, so most of H is zero, and $PH^\top$ skips those zeros.

One EKF step. `correct` applies the ordinary EKF update with some rows of the observation,

$$x = x_0 + K\big(z - h(x_0)\big), \qquad K = PH^\top S^{-1}, \qquad P \leftarrow P - KHP, \tag{14}$$

with $K^\top$ solved from $SK^\top = (PH^\top)^\top$ by Cholesky, and nothing is updated if S is not positive definite. Then R absorbs the orientation error (section 2).

1-point RANSAC. The matches of a frame come with no guarantee, and `EKF::update` decides which to use, as in Civera et al. [2]:

1. *Hypotheses.* For every match j, a copy of the state is updated with that match alone,

$$x^{(j)} = x_0 + K_j\big(z_j - h_j(x_0)\big), \qquad K_j = PH_j^\top S_{jj}^{-1}. \tag{15}$$

   One pixel only fixes two degrees of freedom of the pose, and the others move according to their correlations with those two in P, the most likely way given the prediction, which is enough to tell which other matches agree. The hypotheses are states with $e \neq 0$, projected with $R\exp([e]_\times)$.

2. *Support.* The prediction $h(x^{(j)})$ is computed with the full nonlinear model, and every match within 3 px of it supports the hypothesis. The features in view at $x^{(j)}$ need not be those in view at x_0, so rows are paired through the feature they belong to, not by position, and a match whose feature leaves the image does not support it. The largest support wins, ties broken by the smallest sum of distances.

3. *Low innovation inliers.* The state is updated with the winning support.

4. *High innovation inliers.* A match can be right and disagree with every single-match hypothesis, if the prediction was far off. After the first update the prediction is much better, so h, H and S are computed again, and every other match with a normalized innovation squared below 9.21, the 99% quantile of χ^2_2, is rescued and used in a second update. Updating in two steps is equivalent to updating with both sets at once, up to the linearization.

Since everything goes through h, H, P and σ, nothing in the update knows about poses or images. Each hypothesis costs one evaluation of the observation model, which is fine for a dozen features. With hundreds, a random subset of hypotheses would do, as in Civera et al. [2].

The plain update is enough. The projection is not linear, and an iterated EKF, Gauss–Newton on the same cost, would follow its curvature. Without a line search it oscillates on this cost, and with the single rescued matches of the second stage, which do not fix a pose, it wanders through the directions they leave free. The curvature it would correct is small for the innovations that 1-point RANSAC lets through.

8 The map

`Mapper` keeps the map, at the end of every frame that is not weak.

Adding. While fewer than 12 features are in view, at most two are added per frame:

1. *Where.* A box a quarter of the frame in each dimension is slid over it by half its size, 10 px away from the border, and the positions with no feature in view inside are tried, the farthest from the features first, so that the features keep a roughly constant density over the frame, as in Davison et al. [3], who place the box at random. A fixed order keeps the runs repeatable. A box can be empty because there is nothing to track in it, and then the next one is tried.

2. *Salient.* `goodFeaturesToTrack` proposes up to 10 corners in the box (Shi–Tomasi [8], quality 0.01, at least 5 px apart), 15 px away from the features in view, strongest first.

3. *Textured.* The quality of `goodFeaturesToTrack` is relative to the best corner of the box, so on a blank wall the least blank point passes. A candidate also needs an absolute texture: the root mean square of the gradient of its patch along its least textured direction, the smaller eigenvalue of the gradient matrix of the patch, as in Shi and Tomasi, must be at least 4 gray levels per pixel. On the six sequences, the features below 2, 31% of them, lived a median of 11 frames and were used in 21% of their searches, and those above 4 were used in about 70%. The correlation of the next test cannot tell a blank patch: it divides by its contrast, and what is left is camera noise.

4. *Distinctive.* A candidate is rejected if its patch correlates with some other place within ± 20 px, outside ± 2 px of itself, with a score of 0.8 or more. That keeps the new features off the inner corners of the chessboard, and off repeated texture in general, which the search could not tell apart within its region.

5. *Initial depth.* ρ_0 is the inverse of the median distance of the well located features in view, or 0.02 cm^{-1} (50 cm) if there are none, and $\sigma_\rho = \rho_0$, so the 2σ interval covers everything from a third of that distance to infinity.

6. The feature gets its patch, a descriptor (section 9) and a seed.

Removing. Every frame, each feature searched counts as an attempt, and as a match if the update used it. After 10 attempts, a feature used in less than half of its last 20 is removed from the state, the covariance (`EKF::removeFeatures`), the feature list and the tracker. Only the recent attempts count, so a feature that stops being matched goes within a third of a second, however long it was good. That removes occluded features, features on the edges of objects, whose patch changes with the point of view, and features on things that move. A feature that comes back into view after at least 10 frames away starts its count anew: its failures from before tell little about it now, and a single bad frame on its return would otherwise remove it.

Two kinds of features are never removed: the corners of the pattern, which hold the scale of the map, and the features used in at least 30 frames. A feature that was good for that long is part of the map, and when the camera comes back after a long excursion, it is what closes the loop: its first searches after the return may fail, since the state has drifted, and by the rule above it would be gone before it could pull the state back. Without this, sequence 6 comes back to the board after a minute with nothing left of the first visit but the board itself, whose look-alike corners match a whole square away.

9 Finding the map again

The search only looks near the seeds and the predictions. When the camera gets somewhere else without the images or the filter following it (a kidnap, simulated by cutting frames out of a sequence), nothing there is right. The features are then looked for where the keyframes that look like the frame saw them, as in Klein and Murray [4] (`Keyframes::match`), and if that fails, by their appearance alone, anywhere in the frame, as in Williams et al. [9] (`matchGlobally`).

Keyframes. After every frame that is not weak, with at least 8 features used, the frame is kept as a keyframe if no keyframe is within 5 cm and 8° of its pose: shrunk to 80×60 pixels, blurred with $\sigma = 2.5$ of those pixels, and with its mean removed, along with the pose (`Keyframes::add`). After a weak frame, the frame is shrunk in the same way, and the three keyframes with the smallest mean squared difference to it are tried in turn. Each is first aligned with the frame by the shift, within 6 pixels of the thumbnail, that makes them most alike, since a small turn of the camera shifts the whole image. The well located features, except those of the pattern, are then projected with the pose of the keyframe, shifted, and searched with their patches, warped to that pose, within 10 px. A pose is fitted to those found, and verified, as below for the descriptors.

The global match needs a corner of the frame on the feature, and an ORB descriptor that survives the few pixels between them, and on these sequences only 3 or 4 of its 10 to 14 matches are right, even while tracking is good, of about 9 features in view. The keyframes need neither: they search with the patches that track the features, around where they should be. They need the camera back near a pose it had, though. On sequence 5, right after the second cut, the keyframe that looks most like the frame is 5 cm and 4° from it, while at the third visit of sequence 6 the camera is 9 to 14 cm from every pose it had, and neither method finds the map. Over 15 runs, kidnaps recovered went from 19 to 23 of 30, and the return to the board at frame 836 of sequence 6 is within 10 cm in 13 runs instead of 9.

Descriptors. Every feature gets an ORB descriptor [7] when it is created (`describe`): 256 binary comparisons of pixel pairs in the 31×31 patch around it, rotated by the orientation of the patch, the direction to the centroid of its intensities, which turns with the image. ORB only computes that orientation for the corners it detects itself, so it is computed here. Seen from the side, a feature can be 60 bits away from how it looked when it was created, so while it is tracked its descriptor in the current frame is kept as well if it is more than 30 bits away from all those it has, up to eight (`Mapper::recordAppearance`).

Matching. Up to 1500 corners of the frame are detected as the features were, with `goodFeaturesToTrack`, since ORB's own corners rarely fall on them, and described in the same way. Each descriptor of each well located feature, except the five chessboard corners, which all look alike, is matched to its nearest corner, if it is within 64 bits and clearly nearer, by a ratio of 0.8, than the best corner more than 5 px away from it: corners around the same structure look alike, but are not an ambiguity. A pose is fitted to the matches with P3P and RANSAC, 1000 iterations, inliers within 2 px, and at least six must agree.

Verification. A wrong pose can agree with six descriptors by chance. So the pose is verified with the patches: the features are projected with it and searched, warped to it, within 4 px of their projections, and at least six must be found within 1.5 px.

When it is used. A frame is *weak* when fewer than four features, or fewer than half of those searched, were used by the update. That means the state may be wrong, even if some matches still agree with it: after a kidnap, 1-point RANSAC can keep finding a few look-alikes that agree with a wrong pose.

- After a weak frame, the keyframes or the global match decide: if it finds a pose, `resetCamera` moves the camera there, with no velocity and a large uncertainty (2 cm, 0.2 rad, 30 cm/s, 1.5 rad/s) and no correlation with the features, and the seeds move to their projections.
- While fewer than half of the features searched are used, the map is left alone: no feature is added, since it would be placed with a wrong pose, and no failed match is counted, since the failures are the state's. A frame with fewer than four features used, but most of those searched, is not one of these: the camera is entering a part of the scene with no features yet, and needs new ones. Freezing the map there would keep tracking weak, and the map frozen, for good.

10 Results

Ground truth. The sequences start on the chessboard, and some come back to it. `analyze_pattern` finds the whole board in every frame where it can, and fits the camera pose to its 54 corners. A run is scored in pixels, the distance from the detected corners to the projections of the five chessboard features from the estimated pose (`evaluate_tracking.py`), and by the error of the camera position when the board is seen again. The detector misses the blurriest frames, the fastest ones, so the annotated frames are checked as well.

The runs are chaotic. A tiny difference, such as changing the pixel noise by one part in a billion, can flip a borderline decision, a match accepted or a feature added, and the two runs then diverge. A single run cannot tell a better system from a lucky one. Every number below

pools 15 runs: five sets of initial chessboard corners, each with three negligible changes of the pixel noise.

sequence	result over 15 runs
1	median 0 frames over 5 px, at most 3
2	median 5 frames over 5 px, at most 9
3	median 2 frames over 5 px, never lost
5, back on the board at frames 336, 514, 566	median errors 2.3, 2.1, 2.0 cm
5, kidnaps: frames 141–249 and 421–479 cut	recovered in 6 and 12 runs
6, board visits at frames 359, 411	median errors 4.1, 1.6 cm
6, board visits at frames 771, 836	median errors 6.2, 1.1 cm
6, board visits at frames 1413, 1549	lost in every run

Sequences 1 to 3 look at the board most of the time. Sequences 4 and 5 leave it after the initialization and track natural features, a table, a bowl, a cable, chairs, pictures, with 5 to 20 features per frame, and sequence 5 comes back to the board twice. Sequence 6 is a minute long and comes back three times. The map holds the first time. The second excursion drifts, often by tens of cm when the board comes back into view at frame 771, but the features kept from the first visit close the loop: by frame 836, 13 runs out of 15 are within 10 cm. Before the third visit the camera sweeps a blurred, almost textureless sofa, and tracking is lost in every run; the global match does not find the map again before the end. The losses of sequence 3 come from the same set of initial corners, where the camera swings fast and the board is blurred.

Speed. On sequence 6, while tracking, 6 ms per frame, the median of the averages over every second, against 14 ms with the same code on 640×480 frames. There the filter took 1 to 3 ms, and most of the time went to the images: 6 to 10 ms the optical flow, of which 4 ms was the Lucas–Kanade tracking of the 150 corners forwards and backwards, and up to 3 ms the affine motion fitted around each seed, and 2 to 6 ms the search of the patches. On a quarter of the pixels, with windows and patches half as wide, they cost a fraction of that. Over 15 runs, the accuracy is the same within the spread of the runs: the first return of sequence 5 is 1 cm worse, the second visit of sequence 6 better, and the kidnaps recovered go from 20 to 18 of 30. Three more things keep the time down:

- The masked correlation of the patches is computed from three plain correlations (`maskedCorrelation`), the same score that OpenCV's masked `matchTemplate` gives, 2.7 times faster.
- The pyramid of each frame for the flow of the corners is built once, and used forwards, backwards and again as the previous frame. It is built with the window of the corners: with the larger window of the features it would have one level less, and the corners lose fast motions, sequence 3 in 11 runs out of 15.
- When new features are added, the candidate corners are checked once: the boxes overlap, and are searched again for each feature added, and checking every candidate each time took up to 40 ms in one frame out of 14.

After a weak frame the keyframes and the global matching run, so while the camera is lost a frame takes longer, up to 30 ms on average over a second. Live, a frame that takes longer than the frame period drops the next one, and the interval of the prediction doubles, and with it the uncertainty of the predictions. With `--timing` the decoding of the camera frames is reported too, about 1 ms. Each frame read is a new image that nothing writes to, which tracking, the features and the window share without copying it. The window is drawn and shown by the

main thread, and tracking runs in another, which hands it only its last frame (`Display`): the display took 3 ms per frame at 640×480, and much more with the window maximized, which was enough to drop frames. 150 corners track as well as 300, but on 640×480 frames a 15×15 window with at most 10 iterations lost sequences 2 and 3 in every run.

Limitations.

- At a depth discontinuity, the seed of a feature follows the surface with more corners. Keeping both predictions, its own flow and the motion of its neighbors, and letting the update choose would fix it.
- 12 of the 30 kidnaps are not recovered in time, mostly the first one, where the camera sees, after the cut, a part of the scene it had not seen before. Which ones are recovered changes a lot from run to run: the float rounding of the faster correlation alone moved the first kidnap from 13 recoveries to 9, and the second from 10 to 11, out of 15.
- Over long excursions the map drifts, and the filter does not know: the uncertainty it reports grows far less than the error. A simulation of the filter with exact data association shows the same, so it is the filter and not the matching: new features enter with a very uncertain depth, and every update with them gives the camera information that it does not have. With features entering at their true depth the filter is consistent.
- The features used for long are never removed, so the map, and the cost of the filter, grow with the explored scene.
- The global match describes only detected corners, a few pixels away from where the descriptor of a feature matches best, and ORB descriptors change a lot over a few pixels. On the return to the board at the end of sequence 6, most visible features match their descriptors well within 4 pixels of their true position, but only one or two at the corners described.

11 Correspondence with the code

frame loop, parameters of the filter	`main.cpp`
R, exp, $[\cdot]_\times$	`EKF::orientation`, `rotation`, `skew` in `camera.h`
f, F, W, Q, J_r	`MotionModel` in `ekf.h`, `rightJacobian`
prediction	`EKF::predict`
h, H, S	`EKF::observe`
update, 1-point RANSAC	`EKF::update`, `EKF::correct`
state augmentation, removal	`EKF::addFeature`, `EKF::removeFeatures`
relocalization of the camera	`EKF::resetCamera`
inverse depth	`inverse_depth::toCamera`, `fromPixel`, `fromPoint`, `toPoint`
projection	`CameraModel::projectCameraPoint`
initialization	`initializeFromPattern`
seeds	`FlowTracker` in `tracking.h`
patch search, returning features	`findPatch`, `matchFeatures`
adding, removing features	`Mapper::addFeatures`, `Mapper::removeUnreliable`
descriptors	`describe`, `Mapper::recordAppearance`
keyframes	`Keyframes` in `relocalization.h`
global matching	`matchGlobally` in `relocalization.h`
input, output	`FrameSource`, `StateStream`
window	`Display`, `draw` in `display.h`

References

[1] J. Civera, A. J. Davison and J. M. M. Montiel. Inverse depth parametrization for monocular SLAM. *IEEE Transactions on Robotics*, 24(5):932–945, 2008.

[2] J. Civera, O. G. Grasa, A. J. Davison and J. M. M. Montiel. 1-point RANSAC for extended Kalman filtering: application to real-time structure from motion and visual odometry. *Journal of Field Robotics*, 27(5):609–631, 2010.

[3] A. J. Davison, I. D. Reid, N. D. Molton and O. Stasse. MonoSLAM: real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1052–1067, 2007.

[4] G. Klein and D. Murray. Improving the agility of keyframe-based SLAM. In *Proceedings of the European Conference on Computer Vision*, pages 802–815, 2008.

[5] E. J. Lefferts, F. L. Markley and M. D. Shuster. Kalman filtering for spacecraft attitude estimation. *Journal of Guidance, Control, and Dynamics*, 5(5):417–429, 1982.

[6] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence*, pages 674–679, 1981.

[7] E. Rublee, V. Rabaud, K. Konolige and G. Bradski. ORB: an efficient alternative to SIFT or SURF. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2564–2571, 2011.

[8] J. Shi and C. Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

[9] B. Williams, G. Klein and I. Reid. Automatic relocalization and loop closing for real-time monocular SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(9):1699–1712, 2011.